

1.Mysql基础

1.1 介绍

SQL 是1986年10 月由美国国家标准局（ANSI）通过的数据库语言美国标准，接着，国际标准化组织（ISO）颁布了

SQL正式国际标准。1989年4月，ISO提出了具有完整性特征的SQL89标准，1992年11月又公布了SQL92标准，在此

标准中，把数据库分为三个级别：基本集、标准集和完全集。

关系型数据库：Mysql Oracle sqlServer

mysql 几十M 开源免费得的数据库，体积非常小 集群
Oracle 数据库起家,性能非常好,收费的 非常贵 早期Oracle, 去O (Oracle) 去I (IBM)
Oracle 收购了sun公司,收购了mysql

特性：1、关系型数据库，是指采用了关系模型来组织数据的数据库；

2、关系型数据库的最大特点就是事务的一致性；

3、简单来说，关系模型指的就是二维表格模型，而一个关系型数据库就是由二维表及其之间的联系所组成的一个数据组织。

优点：1、容易理解：二维表结构是非常贴近逻辑世界一个概念，关系模型相对网状、层次等其他模型来说更容易理解；

2、使用方便：通用的SQL语言使得操作关系型数据库非常方便；

3、易于维护：丰富的完整性(实体完整性、参照完整性和用户定义的完整性)大大减低了数据冗余和数据不一致的概率；

4、支持SQL，可用于复杂的查询。

缺点：1、为了维护一致性所付出的巨大代价就是其读写性能比较差；

2、固定的表结构；

3、高并发读写需求；

4、海量数据的高效率读写

1.2 关系数据库基本概念

关系型数据库的基本概念



Student 定义一个类定义一个模板

创建对象 new Student("1","小龙",21);

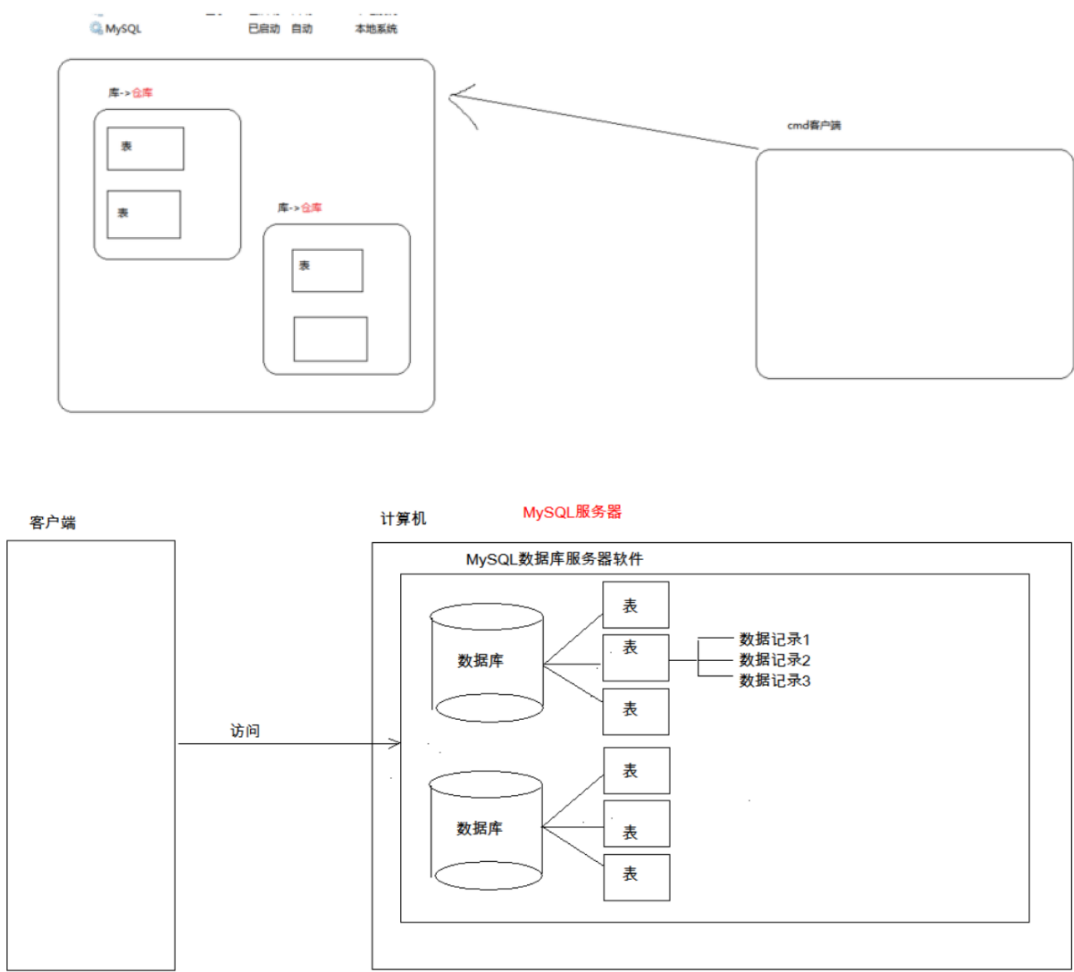


表->类

对象->一条记录

mysql -u root -p

```
Microsoft Windows [版本 10.0.19041.928]
(c) Microsoft Corporation。保留所有权利。

C:\Users\gxa>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 185
Server version: 5.6.40-log MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> _
```

1.3 校对规则

校对规则，就是确定字符如何排序及比较大小

a b c d f

latin2	ISO 8859-2 Central European	latin2_general_ci
swe7	7bit Swedish	swe7_swedish_ci
ascii	US ASCII	ascii_general_ci
ujis	EUC-JP Japanese	ujis_japanese_ci
sjis	Shift-JIS Japanese	sjis_japanese_ci
hebrew	ISO 8859-8 Hebrew	hebrew_general_ci
tis620	TIS620 Thai	tis620_thai_ci
euckr	EUC-KR Korean	euckr_korean_ci
koi8u	KOI8-U Ukrainian	koi8u_general_ci
gb2312	GB2312 Simplified Chinese	gb2312_chinese_ci
greek	ISO 8859-7 Greek	greek_general_ci
cp1250	Windows Central European	cp1250_general_ci
gbk	GBK Simplified Chinese	gbk_chinese_ci
latin5	ISO 8859-9 Turkish	latin5_turkish_ci
armSCII8	ARMSCII-8 Armenian	armSCII8_general_ci
utf8	UTF-8 Unicode	utf8_general_ci
ucs2	UCS-2 Unicode	ucs2_general_ci
cp866	DOS Russian	cp866_general_ci
keybcs2	DOS Kamenicky Czech-Slovak	keybcs2_general_ci
macce	Mac Central European	macce_general_ci
macroman	Mac West European	macroman_general_ci
cp852	DOS Central European	cp852_general_ci

1.4 操作数据库的相关sql

```
-- 进入mysql ***
mysql -u root -p
-- 输入密码

-- 创建一名字为mydb1的数据库 创建数据库的时候没有指定编码，默认会使用安装数据库指定的编码
create database 名字 ***

create database mydb1;

-- 查看所有的库 ***
show databases;

-- 查看前面创建数据库的定义信息
show create database 数据库的名字

show create database mydb1;

-- 创建一名字为mydb2的数据库字符集指定为utf8
create database mydb2 character set utf8;

-- 创建一名字为mydb3的数据库字符集指定为utf8 校对规则 utf8_general_ci
create database mydb3 character set utf8 collate utf8_general_ci;

-- 删除一个数据库 ***
drop database 数据库的名字
drop database mydb1;

-- 改变mydb2的编码为gb2312
alter database 数据库的名字 character set 编码;

alter database mydb2 character set gb2312;
show create database mydb2;
```

1.5 操作表的相关sql

Java中的日期对象

```
public class Demo03 {

    public static void main(String[] args) {

        //birthday 日期
    }
}
```

```

//如果使用无参构造器是获取当前时间
Date date=new Date();

System.out.println(date);

//String toLocaleString()
System.out.println(date.toLocaleString());

//long getTime()
//返回自 1970 年 1 月 1 日 00:00:00 GMT 以来此 Date 对象表示的毫秒数。
System.out.println(date.getTime());

//如果要创建指定时间的日期

//Date(long date)
//分配 Date 对象并初始化此对象，以表示自从标准基准时间（称为“历元（epoch）”，即
1970 年 1 月 1 日 00:00:00 GMT）以来的指定毫秒数。

Date date2=new Date(39206325962011);
System.out.println(date2.toLocaleString());
}
}

```



create table 表名

```

(
    列名 类型,
    列名 类型,
    列名 类型
);

```

java mysql

```

int    int
long   long
double double
String char(长度) char(60) 固定长度大小的字符串
String varchar(长度) varchar(60) 可变长度的字符串
boolean int(1) 只能表示 0和1

```

输入的字符个数没有达到长度时，会默认在空余位置填充空格

输入多少个字符（不超过长度），就占用多少

```
String text 大文本
blob 二进制数据
date date(年月日)
date datetime(年月日 时分秒)
BigDecimal decimal(8,2)
```

java中属性 多个单词组成, 驼峰命名
数据库中列名 多个单词, 多个单词用_隔开

-- 使用数据库

use 数据库的名字;

use mydb2;

```
create table employee
(
    id int,
    name varchar(30),
    sex int(1),
    birthday date,
    entry_date date,
    job varchar(50),
    salary decimal(8,2),
    resume text
);
```

-- 查看库中所有表

show tables;

-- 查看表的创建细节

show create table employee;

-- 查看表结构 ***

desc 表名;

desc employee;

-- 在上面的员工增加image列

alter table 表名 add 列名 类的类型;

alter table employee add image varchar(200);

-- 修改job的长度为60

alter table 表名 modify 列名 类的类型;

alter table employee modify job varchar(60);

-- 修改表的名字

rename table 表原来的名字 to 表新的名字;

rename table employee to user;

rename table user to employee;

-- 修改表的字符集编码

alter table employee character set gb2312;

show create table employee;

修改已有字段的类型
ALTER TABLE emp_my
MODIFY COLUMN MGR
VARCHAR(255);

删除已有的列
ALTER TABLE emp_my
DROP COLUMN SEX;

添加字段(COLUMN可以省)
ALTER TABLE emp_my ADD COLUMN
SEX VARCHAR(50);

添加主键
ALTER TABLE dept_my
ADD CONSTRAINT
pk_dept_my_deptno PRIMARY KEY
(deptno); # 给dept_my设置主键

添加外键
ALTER TABLE emp_my
ADD CONSTRAINT fk_emp_deptno
FOREIGN KEY(deptno)
REFERENCES dept_my(deptno)
ON DELETE CASCADE
ON UPDATE CASCADE;

```
-- 修改name的名字为username
alter table employee change column name username varchar(100);

-- 删除表 ***
drop table 表名;
drop table employee;
```

1.6 插入

insert into 表名(列名1,列名2,列名3) values(值1, 值2, 值3); ***

默认插入的数据默认为NULL

插入的数据日期和字符串要求用"引起来

所有的值加上单引号，一定不会错

```
insert into employee (id,name,sex,birthday,salary,resume) values
(1,'alg',0,'2021-05-10',0.1,'hello');
insert into employee (id,name,sex,birthday,salary,resume) values
('2','alg',0,'2021-05-10',0.1,'hello');
insert into employee (id,name,sex,birthday,salary,resume) values
('3','alg',1,'2021-05-10',0.1,'hello');

-- select * from 表名;
select * from employee; ***

-- 插入数据的时候列和值要匹配的 列名字的个数和类型都要和值匹配
-- 错误写法
insert into employee (id,name,sex,birthday,salary,resume) values
('3','alg',1,1,0.1,'hello');insert into employee
(id,name,sex,birthday,salary,resume) values ('3','alg',1,1,0.1,'hello');

-- 插入所有列的简写方式 不写列名代表跟上了所有的列名 项目不用这种写法
insert into employee values ('6','alg',1,'2021-05-10','2021-05-10','aa',0.1,'hello');

-- 插入中文
insert into employee (id,name,sex,birthday,salary,resume) values ('9','奥利给',1,'2021-05-10',0.1,'hello');

-- 查看mysql的编码
show variables like 'chara%';

set character_set_client=gb2312;

-- 重新插入

-- 显示乱码
set character_set_results=gb2312;
```

1.7 修改

```
-- update 表名 set 列名1=值,列名2=值... where 某个条件

-- 将所有员工的工资改成12000
update employee set salary=12000;

-- 讲奥利给的工资降成了一元
update employee set salary=1 where id=9;

-- 修改奥利给工资为3000元，工作改为程序员鼓励师
update employee set job='程序鼓励师',salary='3000' where id=9;

-- 将奥利给的工资在原来的基础之上增加3000元
update employee set salary=salary+3000 where name='奥利给';
```

← 相关更新语句 记录在数据备份处

1.8 删除

delete from 表名 where 条件

```
-- 删除id为9的记录
delete from employee where id=9;

-- 删除所有记录
delete from employee;

-- truncate 摧毁表结构，重建
truncate table employee;
```

2.简单查询

```
create table student
(
    id int,
    name varchar(20),
    chinese float,
    english float,
    math float
);

insert into student(id,name,chinese,english,math) values(1,'张小明',89,78,90);
```



```
insert into student(id,name,chinese,english,math) values(2,'李进',67,98,56);
insert into student(id,name,chinese,english,math) values(3,'王五',87,78,77);
insert into student(id,name,chinese,english,math) values(4,'李一',88,98,90);
insert into student(id,name,chinese,english,math) values(5,'李来财',82,84,67);
insert into student(id,name,chinese,english,math) values(6,'张进宝',55,85,45);
insert into student(id,name,chinese,english,math) values(7,'黄蓉',75,65,30);
```

2.1 投影操作

select [distinct] 列名1 [as] 别名, 列名2 [as] 别名,列名3 [as] 别名 from 表名 where

[]中的语法是选择使用的语法

```
-- 查询所有学生的数据    *代表显示所有的列

select * from student;

-- 查询所有学生的 id name chinese
select id,name,chinese from student;

-- 查询班上所有的英语成绩    distinct取出重复的结果
select distinct english from student;

-- 查询所有学生的总分
select name,(chinese+math+english) from student;

-- 查询所有学生的总分，在总分加上20分的特长分
select name,(chinese+math+english)+20 from student;

-- 使用别名
select name as 姓名,(chinese+math+english)+20 as 总分 from student;
select name 姓名,(chinese+math+english)+20 总分 from student;
```

SELECT DISTINCT 字段A
FROM 表; 去除表中字段A重复的数据

2.2 选择操作

```
-- 查询王五同学的成绩
select * from student where name='王五';

-- 查询英语成绩大于80分的同学
select * from student where english>80;

-- 查询总成绩大于200分的同学

select * from student where (chinese+math+english)>200;

select name as 姓名,(chinese+math+english)+20 as 总分 from student where
(chinese+math+english)>200;
```

-- 查询数学成绩在70分到90分的同学
`select name,math from student where math>=70 and math<=90;`

-- where 列 between 值1 and 值2; 包含边界
`select name,math from student where math>=70 and math<=90;`
==
`select name,math from student where math between 70 and 90;`

-- 查询数学成绩不等于90分的同学 <> 不等于
`select name,math from student where math != 90;`
`select name,math from student where math <> 90;`

-- 查询语文成绩88分, 英语成绩98分的同学的成绩
`select * from student where chinese = 88 and english =98;`

-- 查询英语成绩为78分或者数学成绩为77分的同学
`select * from student where english = 78 or math =77;`

-- 查询数学成绩90,77,67分的同学

-- 列名 in(值1,值2,值3) 符合这些值的结果会被查询出来

`select * from student where math in(90,77,67);`

<====>
`SELECT * FROM student WHERE
math=90 OR math=77 OR math=67`

-- 列名 not in(值1,值2,值3) 不符合这些值的结果会被查询出来

`select * from student where math not in(90,77,67);`

-- 模糊查询 like %代表任意的0个或者多个字符 _代表任意的一个字符

-- 查询所有姓李的同学的成绩 %代表任意的0个或者多个字符

`select * from student where name like '李%';`

-- 查询三个字姓李的同学的成绩 _代表任意的一个字符

`select * from student where name like '李__';`

-- 查询数学成绩为空的的同学的成绩

`insert into student(id,name,chinese,english) values(8,'奥利给',99,150);`

`select * from student where math is null;`

`select * from student where math is not null;`

选择查询的条数
查询emp表, 从第0行开始, 查询5条数据
`SELECT * FROM emp LIMIT 0,5;`

2.3 排序

`select * from 表名 where 条件 order by 列名1,列名2 ...`

```
-- 按照数学成绩排序输出 默认是升序 asc 升序 desc降序
select name,math from student order by math;

select name,math from student order by math asc;

select name,math from student order by math desc;

-- 按照主要条件英语成绩排序和次要条件数学成绩进行排序
select name,english,math from student order by english desc,math desc;

-- 对李姓同学按照总成绩进行排序

select name 姓名,(chinese+math+english) as 总成绩 from student where name like
'李%' order by 总成绩 desc;
```

如果写出这样

SELECT name,english,math FROM student ORDER BY english, math DESC; 表示先按照english升序, english相同按照math降序 desc 或者 asc 只作用一个字段, 且满足就近原则

2.4 聚合操作

聚合函数（分组函数），这种可以进行函数嵌套使用。
特别注意：使用的字段为空值（=NULL）时，函数会忽略该数据

```
-- 统计班上的学生 count(列名) 计算总共有多少行数据 如果某个字段的记录是null, 不会统计

select count(name) from student;

select count(english) from student;

select count(math) from student;

select count(*) from student;

-- 统计班上数学成绩高于80分的同学

select count(*) as 人数 from student where math>80;

-- 统计总分数大于200分同学的成绩
select count(*) 人数 from student where (chinese+math+english)>200;

-- sum 求和
-- 统计一个班的数学总成绩
select sum(math) from student;

-- 统计各科总成绩
select sum(math),sum(chinese),sum(english) from student;

-- 统计一个所有总分加起来的总分
select sum(chinese+math+english) from student;

-- 统计班上总成绩的平均分 avg平均数

select sum(chinese+math+english)/count(*) from student;

select avg(chinese+math+english) from student;

-- 数据为空会对avg的结果产生影响
select sum(math)/count(*) from student;
```

WHERE 中不能直接写聚合函数, 可以写子查询

```
select avg(math)/count(*) from student;
```

-- 求数学的最高分

```
select max(math) from student;
```

-- 求最低分 不包括空值

```
select min(math) from student;
```

-- 求一个班上总成绩的最高分和最低分

```
select max(chinese+math+english),min(chinese+math+english) from student;
```

如果需要将空值处理0
IFNULL(字段,0) 【MySQL 常用】
ISNULL(字段,0) 【SQL Server 常用】
COALESCE(字段,0) 【通用】

2.5 分组

规则：1. 先确定select后面的字段（想要展示的字段）
2. select后面的字段，如果没有出现在分组函数中，那一定要在group by后面说明

```
create table orders
```

```
(
```

```
id int,
```

```
product varchar(20),
```

```
price float
```

```
);
```

```
insert into orders(id,product,price) values(1,'电视',900);
```

```
insert into orders(id,product,price) values(2,'洗衣机',100);
```

```
insert into orders(id,product,price) values(3,'洗衣粉',90);
```

```
insert into orders(id,product,price) values(4,'桔子',9);
```

```
insert into orders(id,product,price) values(5,'洗衣粉',90);
```

-- 对订单按照商品名称进行分组，每一类商品的总价

```
select product,sum(price) from orders group by product;
```

对分组函数结果继续添加条件，需要在后面使用having。有having 就必须有group by

-- 对订单按照商品名称进行分组，每一类商品的总价，并且要求总价大于100 orders group 后面不能跟where，用having替换

```
select product,sum(price) from orders group by product having sum(price)>100;
```

as：为展示字段取别名

将sum(price) 换成 sumPrice 展示出来

```
SELECT product,sum(price) AS sumPrice FROM orders GROUP BY product HAVING sum(price) > 300
```

3.约束



3.1 主键和主键约束

数据库针对在开发中非法操作主键的这种情况可以添加一种约束，这种约束叫做主键约束

主键约束不等于主键,只是给主键增加约束而已

```
create table student(  
    id int primary key,  
    name varchar(60)  
);  
  
insert into student(id,name) values(1,'奥利给');  
select * from student;  
  
insert into student(id,name) values(1,'奥利给');//报错  
insert into student(id,name) values(null,'奥利给');// Column 'id' cannot be null  
  
-- 加上主键约束后,主键这一列添加数据的时候要求不能为空,而且必须唯一  
-- 一张表只能有一个主键  
  
-- 定义自动增长  
  
create table student  
(  
    id int primary key auto_increment,  
    name varchar(60)  
);  
insert into student(name) values('奥利给');
```

3.2 唯一约束

唯一,但能为空

unique 唯一约束 修饰的列可以唯一
unique可以加在多个字段上面
unique可以为null

```
create table student  
(  
    id int primary key auto_increment,  
    name varchar(60) unique  
);  
insert into student(name) values('奥利给');
```

3.3 定义非空约束

非空

```
create table student
(
    id int primary key auto_increment,
    name varchar(60) unique not null
);
insert into student(name) values('奥利给');
```

主键(效果)=唯一约束+非空约束

3.4 外键约束

如果有外键这一列，但是可以插入非法数据，针对外键插入的数据进行约束

约束有两种方式，一种是我们可以通过java代码在业务上进行限定，另外一种是通过数据库的外键约束

外键约束不等于外键

传统项目一般会使用外键约束

互联网项目不会使用数据库的外键约束,外键约束性能太低

```
-- constraint 外键约束的名字 foreign key (外键列) references 另一张表(主键)

create table husband(
    id int primary key auto_increment,
    name varchar(60)
);

create table wife(
    id int primary key auto_increment,
    name varchar(60),
    husband_id int,
    constraint husband_id_FK foreign key (husband_id) references husband(id)
);

insert into husband(name) values('奥利给');

insert into wife(name,husband_id) values('迪丽热巴',3);
```

4.数据备份

4.1 数据备份

cmd命令



mysqldump -uroot -p 数据库的名字>磁盘路径\sql文件名.sql

```
mysqldump -uroot -p jdbc>E:\jdbc.sql
```

4.2 数据的恢复

sql命令

source sql文件的路径

```
create database jdbc;  
  
use jdbc;  
  
source E:\jdbc.sql
```

cmd命令

mysql -uroot -p 数据库的名字<sql文件的路径

```
mysql -uroot -p jdbc<E:\jdbc.sql  
输入密码
```

5.多表

站在双向角度辨别关系

1、表的关系

A、一对一：

①、辨别：

站在人的角度：1(人):1 (身份证)

站在身份证角度：1 (身份证) :1 (人)

总结：站在双向来讲都是 1:1 结果才是1:1

②、设计：

人 与 身份证

A、主键关联

	t_person	
id	name	birthdate
510104xx	张三	xx

t_card		
id	fzjg	invalid_time
510104xx	青羊区公安局	xx

B、**外键关联**

t_person			
id	name	birthdate	card_id
1	张三	xx	510104xx

t_card		
id	fzjg	invalid_time
510104xx	青羊区公安局	xx

B、一对多：

学生与班级

①、辨别：

站在学生角度：1（学生）：1（班级）

站在班级角度：1（班级）：n（学生）

总结：站在双向来讲都是 1:1 和 1:n 结果才是1:n

②、设计：（外键关联-----**在多的的一方拥有1的一方的外键**）

t_student				
id	name	birth_date	gender	class_id
1	张总	xx	男	10

t_class		
id	name	capacity
10	java	40

C、多对多：

学生和课程

①、辨别：

站在学生角度：1（学生）：n（课程）

站在课程角度：1（课程）：n（学生）

总结：站在双向来讲都是 1:n 结果才是n:n

②、设计（中间表）：

引入中间表

联合主键——不设置主键，只用多个外键组合起一个主键效果

t_student			
id	name	birth_date	gender
1	张总	xx	男

t_course	
id	name
10	java
20	c
30	大数据

t_student_course		
id	student_id	course_id
1	1	10
2	1	IFNULL(10, 30)

5.1 一对多的关系设计



```

create table department(
  id int primary key auto_increment,
  name varchar(60)
);

create table employee(
  id int primary key auto_increment,
  name varchar(60),
  salary decimal(8,2),
  department_id int,
  CONSTRAINT department_id_FK FOREIGN KEY (department_id) REFERENCES department(id)
);
  
```

```

-- 部门表
create table department(
  id int primary key auto_increment,
  name varchar(60)
);

-- 员工表
  
```

```
create table employee(
    id int primary key auto_increment,
    name varchar(60),
    salary decimal(8,2),
    department_id int,
    CONSTRAINT department_id_FK FOREIGN KEY (department_id) REFERENCES
department(id)
);
```

外键约束不等于外键，可以通过java代码的方式来限制外键

如果加上了外键约束:

我们要删除一的方，如果多的一方记住了一的一方的主键，一的一方不能删除。

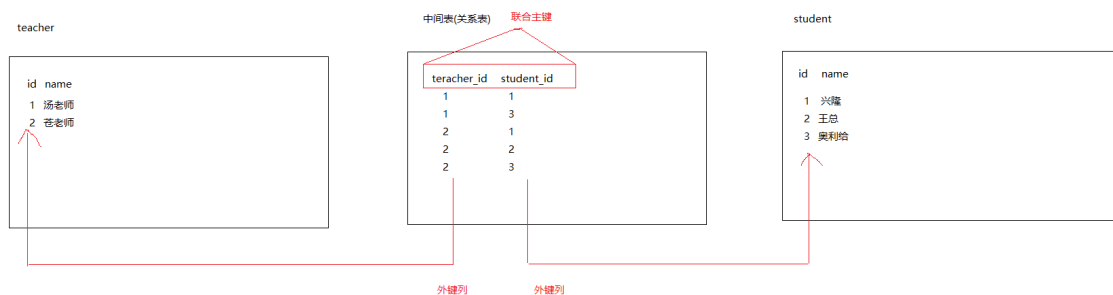
往多的一方插入的外键列插入数据，外键列的id必须来源于一的一方的主键的id

1.级联删除 ON DELETE CASCADE

1.级联更新 ON UPDATE CASCADE

5.2 多对多的关系设计

多对多的表设计：首先新建一张中间表，中间表有两个外键，分别来至于对方表的主键，并且这两个外键要建立联合主键。



```
create table teacher (
    id INT PRIMARY KEY auto_increment,
    name VARCHAR (50)
);

create table student (
    id INT PRIMARY KEY auto_increment,
    name VARCHAR (50)
);

create table teacher_student (
    teacher_id int,
    student_id int,
    primary key(teacher_id,student_id),
```

```

CONSTRAINT teacher_id_FK FOREIGN KEY (teacher_id) REFERENCES teacher(id),
CONSTRAINT student_id_FK FOREIGN KEY (student_id) REFERENCES student(id) );
)

```

5.3 一对一的表设计

方案一:



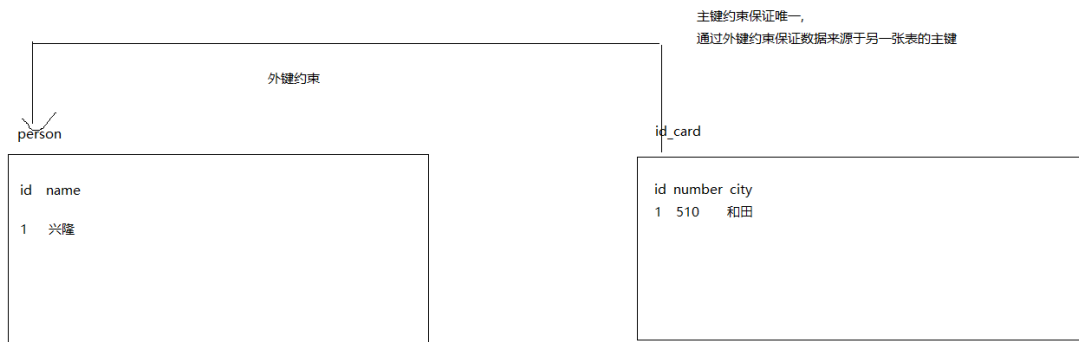
```

create table person(
    id int primary key auto_increment,
    name varchar(60)
);

create table id_card(
    id_card varchar(200) primary key,
    name varchar(60),
    city varchar(60),
    birthday date,
    person_id int unique,
    CONSTRAINT person_id_FK FOREIGN KEY (person_id) REFERENCES person(id)
);

```

方式二



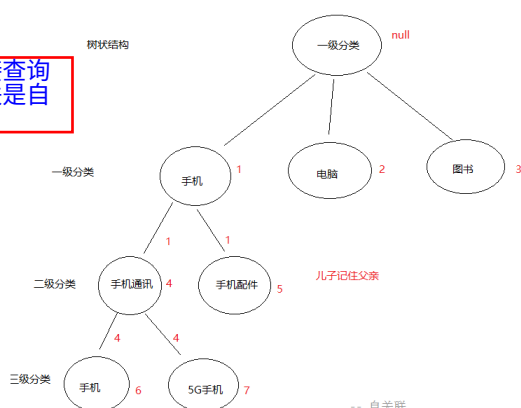
```
create table person(
    id int primary key auto_increment,
    name varchar(60)
);

create table id_card(
    id int primary key,
    number varchar(200),
    city varchar(60),
    CONSTRAINT id_FK FOREIGN KEY (id) REFERENCES person(id)
);
```

5.4 自关联

使用查询时，应该先看
表中有多少条数据，如
果查询结果超过这个数
据太多，就应该警觉。
—— 写完应该核查数据

使用的语法和连接查询
一样，只是自关联是自
己查自己



parent_id记住父节点的id，来源于父亲这条记录的id

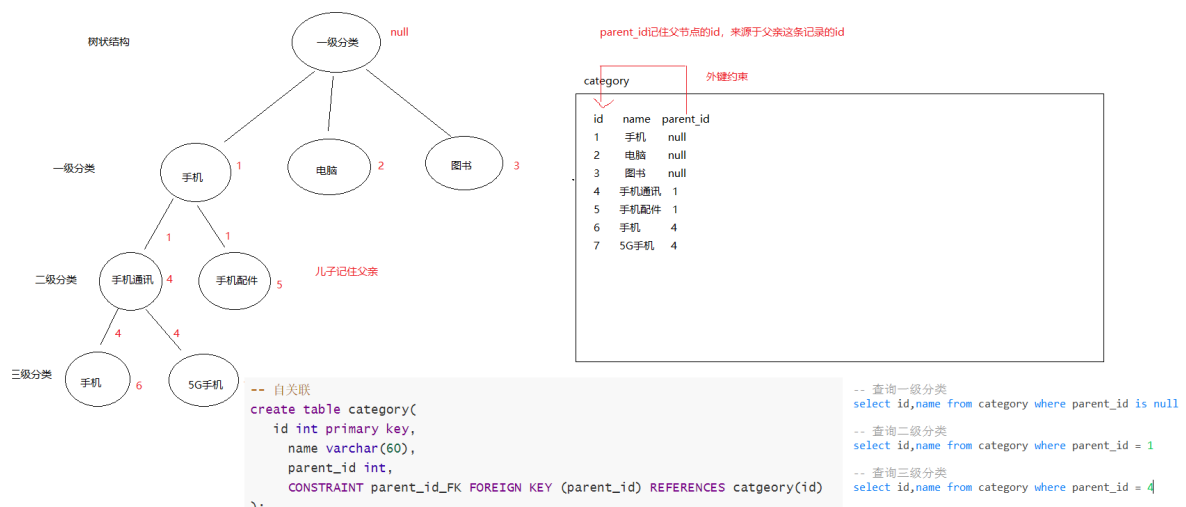
id	name	parent_id
1	手机	null
2	电脑	null
3	图书	null
4	手机通讯	1
5	手机配件	1
6	手机	4
7	5G手机	4

```
-- 自关联
create table category(
    id int primary key,
    name varchar(60),
    parent_id int,
    CONSTRAINT parent_id_FK FOREIGN KEY (parent_id) REFERENCES category(id)
);
```

```
-- 自关联
create table category(
  id int primary key,
  name varchar(60),
  parent_id int,
  CONSTRAINT parent_id_FK FOREIGN KEY (parent_id) REFERENCES category(id)
);
```

就是自己查自己。然后
结果组合

5.5 自关联查询



```
-- 查询一级分类
select id,name from category where parent_id is null

-- 查询二级分类
select id,name from category where parent_id = 1

-- 查询三级分类
select id,name from category where parent_id = 4
```

分类表如何设计:

id	name	p_id
1	手机	
2	手机配件	1
3	手机通讯	1

id	name
10	手机
1001	手机配件
1002	手机通讯

5.6 笛卡尔积

了解，出现在查询条件不合逻辑时

表A

id	name
a1	aa1
a2	aa2

表b

id	name
b1	bb1
b2	bb2
b3	bb3

```
select * from A,B;  返回的结果就是笛卡尔积  2*3=6
```

a1	aa1	b1	bb1
a1	aa1	b2	bb2
a1	aa1	b3	bb3
a2	aa2	b1	bb1
a2	aa2	b2	bb2
a2	aa2	b3	bb3

-- 创建部门表

```
CREATE TABLE dept(  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    NAME VARCHAR(20)  
);
```

```
INSERT INTO dept (NAME) VALUES ('开发部'),('市场部'),('财务部');
```

-- 创建员工表

```
CREATE TABLE emp (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    NAME VARCHAR(10),  
    gender CHAR(1), -- 性别  
    salary DOUBLE, -- 工资  
    join_date DATE, -- 入职日期  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES dept(id) -- 外键，关联部门表(部门表的主键)  
);
```

```
INSERT INTO emp(NAME,gender,salary,join_date,dept_id) VALUES('孙悟空','男',7200,'2013-02-24',1);
```

```
INSERT INTO emp(NAME,gender,salary,join_date,dept_id) VALUES('猪八戒','男',3600,'2010-12-02',2);
```

```
INSERT INTO emp(NAME,gender,salary,join_date,dept_id) VALUES('唐僧','男',9000,'2008-08-08',2);
```

```
INSERT INTO emp(NAME,gender,salary,join_date,dept_id) VALUES('白骨精','女',5000,'2015-10-07',3);
```

```
INSERT INTO emp(NAME,gender,salary,join_date,dept_id) VALUES('蜘蛛精','女',4500,'2011-03-14',1);
```

```
select * from dept,emp;
```

信息

结果 1

剖析

状态

id	NAME	id(1)	NAME(1)	gender	salary	join_date	dept_id
1	开发部	1	孙悟空	男	7200	2013-02-24	1
2	市场部	1	孙悟空	男	7200	2013-02-24	1
3	财务部	1	孙悟空	男	7200	2013-02-24	1
1	开发部	2	猪八戒	男	3600	2010-12-02	2
2	市场部	2	猪八戒	男	3600	2010-12-02	2
3	财务部	2	猪八戒	男	3600	2010-12-02	2
1	开发部	3	唐僧	男	9000	2008-08-08	2
2	市场部	3	唐僧	男	9000	2008-08-08	2

+

-

↶

↷

✕

📄

select * from dept,emp

只读

查询时间: 0.014s

第 4 条记录 (共 15 条)

5.7 连接查询

union 与 union all: union会去掉重复的

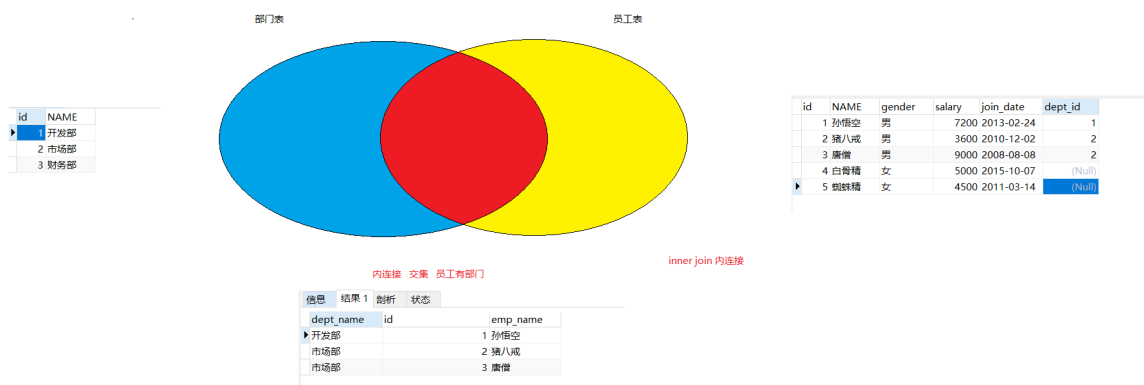
union与union all都是取并集，但是union会去掉重复的数据

```
select empno,ename,job from t_emp
union all
select empno,ename,job from t_emp_bak
```

```
select empno,ename,job from t_emp
union
select empno,ename,job from t_emp_bak
```

5.7.1 内连接

内连接：只保留 两边都能匹配上 的数据外连接：以某张表为主，主表数据全部保留，匹配不上的补 NULL



-- 显示内连接

```
select dept.name as dept_name, emp.id, emp.name as emp_name from dept inner join  
emp on emp.dept_id=dept.id
```

inner join 简写成
join

-- 隐式内连接

```
select * from dept, emp where emp.dept_id=dept.id
```

5.7.2 左外连接

outer 外连接关键字
(可以省略不写)
inner 内连接关键字

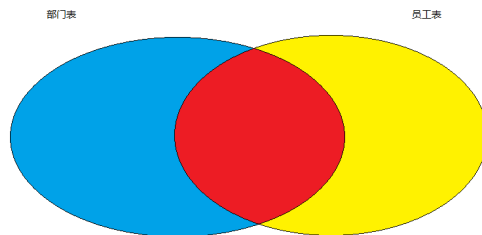
-- 左外连接

```
select dept.name as dept_name, emp.id, emp.name as emp_name from dept left outer  
join emp on emp.dept_id=dept.id
```

-- outer可以省略不写

如果left join左边表
中数据没有匹配上, 也
会将没有匹配上的数据
展示出来

id	NAME
1	开发部
2	市场部
3	财务部



左外连接:
蓝色+红色

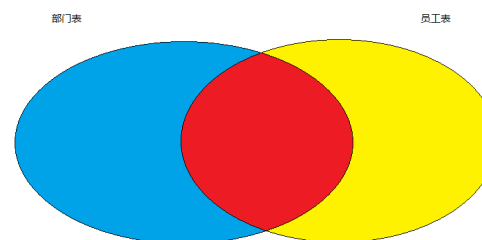
dept_name	id	emp_name
开发部		1 孙悟空
市场部		2 猪八戒
市场部		3 唐僧
财务部		(Null) (Null)

id	NAME	gender	salary	join_date	dept_id
1	孙悟空	男	7200	2013-02-24	1
2	猪八戒	男	3600	2010-12-02	2
3	唐僧	男	9000	2008-08-08	2
4	白骨精	女	5000	2015-10-07	(Null)
5	蜘蛛精	女	4500	2011-03-14	(Null)

5.7.3 右外连接

如果right join右边表
中数据没有匹配上, 也
会将没有匹配上的数据
展示出来

id	NAME
1	开发部
2	市场部
3	财务部



right outer join

dept_name	id	emp_name
开发部		1 孙悟空
市场部		2 猪八戒
市场部		3 唐僧
(Null)		4 白骨精
(Null)		5 蜘蛛精

id	NAME	gender	salary	join_date	dept_id
1	孙悟空	男	7200	2013-02-24	1
2	猪八戒	男	3600	2010-12-02	2
3	唐僧	男	9000	2008-08-08	2
4	白骨精	女	5000	2015-10-07	(Null)
5	蜘蛛精	女	4500	2011-03-14	(Null)

5.8 子查询

查询中嵌套查询，称之为子查询

把一条sql的结果作为另一条sql的条件

互联网项目中子查询尽量少用，效率低

最多3层

```
-- 查询最高工资
select max(salary) from emp;

-- 查询工资等于最高工资的员工的信息 你怎么知道最高工资是多少?
select * from emp where salary =9000;

-- 一条sql，搞定
select * from emp where salary = (select max(salary) from emp);

-- 查询员工的平均工资
select avg(salary) from emp;

-- 嵌套的sql只有一个值的情况 > >= = < <= != 单行单列
-- 查询工资大于平均工资的员工的信息
select * from emp where salary>(select avg(salary) from emp)

-- 嵌套的sql的结果是多行单列 in

-- 查询开发部和市场部的信息

select * from dept where name='开发部' or name='市场部'

-- 查询开发部和市场部的员工信息
select * from emp where emp.dept_id in(1,2)

-- 一条sql子查询sql搞定

select * from emp where emp.dept_id in(select dept.id from dept where name='开发部' or name='市场部');

-- 查询开发部和市场部的员工信息
select emp.name as ename,emp.salary,dept.name as dname from emp,dept where emp.dept_id=dept.id and (dept.name='市场部' or dept.name='开发部');
```

6.索引

mysql引擎: innodb

索引: B+树

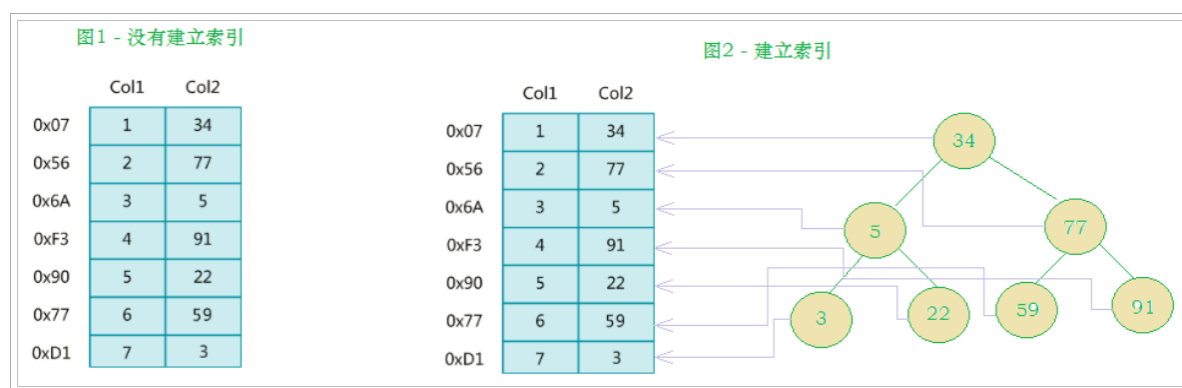
索引一定是排好序的

sql的优化

类似目录

6.1 索引介绍

MySQL官方对索引的定义为: 索引(index)是帮助MySQL高效获取数据的数据结构(有序)。在数据之外, 数据库系统还维护着满足特定查找算法的数据结构, 这些数据结构以某种方式引用(指向)数据, 这样就可以在这些数据结构上实现高级查找算法, 这种数据结构就是索引。如下面的==示意图==所示:



左边是数据表, 一共有两列七条记录, 最左边的是数据记录的物理地址(注意逻辑上相邻的记录在磁盘上也并不是一定物理相邻的)。为了加快Col2的查找, 可以维护一个右边所示的二叉查找树, 每个节点分别包含索引键值和一个指向对应数据记录物理地址的指针, 这样就可以运用二叉查找快速获取到相应数据。

一般来说索引本身也很大, 不可能全部存储在内存中, 因此索引往往以索引文件的形式存储在磁盘上。索引是数据库中用来提高性能的最常用的工具。

6.2 索引优势劣势

优势

- 1) 类似于书籍的目录索引, 提高数据检索的效率, 降低数据库的IO成本。
- 2) 通过索引列对数据进行排序, 降低数据排序的成本, 降低CPU的消耗。

劣势

- 1) 实际上索引也是一张表, 该表中保存了主键与索引字段, 并指向实体类的记录, 所以索引列也是要占用空间的。
- 2) 虽然索引大大提高了查询效率, 同时却也降低更新表的速度, 如对表进行INSERT、UPDATE、DELETE。因为更新表时, MySQL 不仅要保存数据, 还要保存一下索引文件每次更新添加了索引列的字段, 都会调整因为更新所带来的键值变化后的索引信息。

6.3 索引结构

索引是在MySQL的存储引擎层中实现的，而不是在服务器层实现的。所以每种存储引擎的索引都不一定完全相同，也不是所有的存储引擎都支持所有的索引类型的。MySQL目前提供了以下4种索引：

- BTREE 索引：最常见的索引类型，大部分索引都支持 B 树索引。
- HASH 索引：只有Memory引擎支持，使用场景简单。
- R-tree 索引（空间索引）：空间索引是MyISAM引擎的一个特殊索引类型，主要用于地理空间数据类型，通常使用较少，不做特别介绍。
- Full-text（全文索引）：全文索引也是MyISAM的一个特殊索引类型，主要用于全文索引，InnoDB从Mysql5.6版本开始支持全文索引。

MyISAM、InnoDB、Memory三种存储引擎对各种索引类型的支持

InnoDB引擎 支持事务的 

MyISAM引擎 不支持事务

不支持事务、只有表级锁

Memory引擎 保存在内存中

索引	InnoDB引擎	MyISAM引擎	Memory引擎
BTREE索引	支持	支持	支持
HASH 索引	不支持	不支持	支持
R-tree 索引	不支持	支持	不支持
Full-text	不支持	支持	不支持

我们平常所说的索引，如果没有特别指明，都是指B+树（多路搜索树，并不一定是二叉的）结构组织的索引。其中聚集索引、复合索引、前缀索引、唯一索引默认都是使用 B+tree 索引，统称为 索引。

6.4 索引分类

联合索引(复合索引)：多个字段作为索引 和 数据 聚集在一起
特点：在排序时，先更根据字段1排序，字段1相同使用字段2排序...

1) 主键索引：设定为主键后数据库会自动建立索引，innodb为聚簇索引

```
-- 随表一起建索引：
CREATE TABLE customer (
  id INT(10) UNSIGNED AUTO_INCREMENT ,
  customer_no VARCHAR(200),
  customer_name VARCHAR(200),
  PRIMARY KEY(id)
);

-- 使用AUTO_INCREMENT关键字的列必须有索引(只要有索引就行)。
CREATE TABLE customer2 (id INT(10) UNSIGNED,customer_no
VARCHAR(200),customer_name VARCHAR(200),
  PRIMARY KEY(id)
);

-- 单独建主键索引：
ALTER TABLE customer add PRIMARY KEY customer(customer_no);

-- 删除建主键索引：
ALTER TABLE customer drop PRIMARY KEY ;

-- 修改建主键索引：
-- 必须先删除掉(drop)原索引，再新建(add)索引
```

创建普通索引
CREATE INDEX 索引名 ON 表(字段)

查看索引

show index from table_name;

还可以通过查看执行计划判断是否使用索引
EXPLAIN SELECT * FROM table_name WHERE id=1; (explain + sql查询语句，通过观察结果表中的key字段进行判断)

索引一般是在WHERE查询时使用

2) 单值索引(普通索引)：即一个索引只包含单个列，一个表可以有多个单列索引

#随表一起建索引：

```
CREATE TABLE customer (  
    id INT(10) UNSIGNED AUTO_INCREMENT ,  
    customer_no VARCHAR(200),  
    customer_name VARCHAR(200),  
    PRIMARY KEY(id),  
    KEY (customer_name)  
);
```

#随表一起建立的索引 索引名同 列名(customer_name)

#单独建单值索引：

```
CREATE INDEX idx_customer_name ON customer(customer_name);
```

#删除索引：

```
DROP INDEX idx_customer_name ;
```

3) 唯一索引：索引列的值必须唯一，但允许有空值 唯一约束

#随表一起建索引：

```
CREATE TABLE customer (  
    id INT(10) UNSIGNED AUTO_INCREMENT ,  
    customer_no VARCHAR(200),  
    customer_name VARCHAR(200),  
    PRIMARY KEY(id),  
    KEY (customer_name),  
    UNIQUE (customer_no)  
);
```

#建立 唯一索引时必须保证所有的值是唯一的（除了null），若有重复数据，会报错。

#单独建唯一索引：

```
CREATE UNIQUE INDEX idx_customer_no ON customer(customer_no);
```

#删除索引：

```
DROP INDEX idx_customer_no on customer ;
```

4) 复合索引：即一个索引包含多个列

#随表一起建索引：

```
CREATE TABLE customer (  
    id INT(10) UNSIGNED AUTO_INCREMENT ,  
    customer_no VARCHAR(200),  
    customer_name VARCHAR(200),  
    PRIMARY KEY(id),  
    KEY (customer_name),  
    UNIQUE (customer_name),  
    KEY (customer_no,customer_name)  
);
```

#单独建索引:

```
CREATE INDEX idx_no_name ON customer(customer_no,customer_name);
```

#删除索引:

```
DROP INDEX idx_no_name ON customer ;
```

5) MySQL全文检索是利用查询关键字和查询列内容之间的相关度进行检索,可以利用全文索引来提高匹配的速度。

InnoDB不支持

MyISAM不会用

lunnce 全文检索

solr

Elasticsearch 实时搜索 地位位置搜索 天生支持集群

全文检索

中国人

分词

中国 中国人 人

华为

```
product
id
name 商品的名字
title 商品标题
```

7.事务

保证一系列操作同时成功或者同时失败,不能出现一半一半失败的情况。

7.1 事务的引入

需求:

王老板要给小汤转账工资 转账1000

对象 account @jdbc (localhost) - 表		
开始事务 文本 筛选 排序 导入 导出		
id	name	money
1	wlb	3000
2	alg	2000

```
-- 王老板的操作

set session transaction isolation level read uncommitted;

select * from account;

start transaction;

update account set money=money-1000 where id=1;
update account set money=money+1000 where id=2;

select * from account;

-- 打电话给奥利给

奥利给查询账户金额

set session transaction isolation level read uncommitted;
select * from account;

-- 王老板
rollback;

update account set money=money-1000 where id=1;
update account set money=mony+1000 where id=2;

要么都成功，要么都失败
```

7.2 事务的概述

事务指的是逻辑上的一组操作(多条sql语句),组成这组操作的各个单元要么全都成功,要么全都失败.

事务作用：保证在一个事务中多次操作要么全都成功,要么全都失败.

例如转账：

update account set money=money-100 where name='tom';//tom转出100块

update account set money=money+100 where name='jerry';//jerry收到100块

以上两条sql语句,很明显是一组语句,因为转账要么都成功要么都失败,不应该出现一方成功另一方失败的情况.那么我

们就需要把他们看成一个事务

7.3 mysql事务操作

mysql是默认自动提交,
开启事务需要关闭自动
提交

sql语句	描述
start transaction;	开启事务
commit;	提交事务
rollback;	回滚事务

```
-- 开启事务
start transaction;

select * from account;

update account set money=money-1000 where id=1;
update account set money=money+1000 where id=2;

-- 提交事务 事务一旦提交才会生效，不提交就不会生效
commit;

-- 回滚到事务开启的状态
rollback;
```

7.4 事务的特性

ACID

原子性：强调事务的不可分割.多条语句要么都成功，要么都失败。

事务最小的单位

一致性：强调的是事务的执行的前后，数据要保持一致。

隔离性：一个事务的执行不应该受到其他事务的干扰。

也得不到其他事务的数据

持久性：事务一旦结束(提交/回滚)数据就持久保持到了数据库。

MySQL数据库中事务默认自动提交

* 事务提交的两种方式:

* 自动提交:

- * **mysql**就是自动提交的
- * 一条DML(增删改)语句会自动提交一次事务。

* 手动提交:

- * **Oracle** 数据库默认是手动提交事务
- * 需要先开启事务,再提交

* 修改事务的默认提交方式:

- * 查看事务的默认提交方式: **SELECT @@autocommit;** -- 1 代表自动提交 0 代表手动提交
- * 修改默认提交方式: **set @@autocommit = 0;**

7.5 事务的隔离级别

-- 查询隔离级别

```
select @@tx_isolation;
```

-- 设置隔离级别

```
set transaction isolation level Read uncommitted;
```

数据库共定义了四种隔离级别:

Serializable: 可避免脏读、不可重复读、虚读情况的发生。(串行化)

Repeatable read: 可避免脏读、不可重复读情况的发生。(可重复读)(mysql默认)

Read committed: 可避免脏读情况发生(读已提交)。

Read uncommitted: 最低级别,以上情况均无法保证。(读未提交)

脏读:

指一个事务读取了另外一个事务未提交的数据。

不可重复读:

在一个事务内读取表中的某一行数据,多次读取结果不同。例如银行想查询A帐户余额,第一次查询A帐户为200元,此时A向帐户存了100元并提交了,银行接着又进行了一次查询,此时A帐户为300元了。银行两次查询不一致,可能就会很困惑,不知道哪次查询是准的。和脏读的区别是,脏读是读取前一事务未提交的脏数据,不可重复读是重新读取了前一事务已提交的数据。很多人认为这种情况就对了,无须困惑,当然是后面的为准。我们可以考虑这样一种情况,比如银行程序需要将查询结果分别输出到电脑屏幕和写到文件中,结果在一个事务中针对输出的目的地,进行的两次查询不一致,导致文件和屏幕中的结果不一致,银行工作人员就不知道以哪个为准了。

虚读(幻读)是指在一个事务内读取到了别的事务插入的数据,导致前后读取不一致。如丙存款100元未提交,这时银行做报表统计account表中所有用户的总额为500元,然后丙提交了,这时银行再统计发现帐户为600元了,造成虚读同样会使银行不知所措,到底以哪个为准。

//设置read uncommitted级别:

```
set session transaction isolation level read uncommitted; 读未提交
```



```
//设置read committed级别: 可以解决脏读
set session transaction isolation level read committed; 读已提交

//设置repeatable read级别: 可以解决不可重复读和脏读 但是可能会发生幻读 (默认)
set session transaction isolation level repeatable read; 可重复读

//设置serializable级别:
set session transaction isolation level serializable; 串行化

select @@tx_isolation 查询当前事务隔离级别
```

7.6 隔离级别演示

mysql中怎么查看隔离级别

```
全局的: mysql8
      select @@global.transaction_isolation;
mysql5:
      select @@global.tx_isolation;
当前session:
      mysql8:
          select @@transaction_isolation;
      mysql5:
          select @@tx_isolation;
修改当前session的隔离级别:
      set session transaction isolation level;
```

开启事务:

```
start transaction;
```

提交事务:

```
commit;
```

回滚事务:

```
rollback;
```

自动提交:

```
set autocommit=0 不自动提交
```

- read-uncommitted

mysql中的innodb引擎默认隔离级别: REPEATABLE-READ

1、演示脏读 一个事务 读到 另外一个事务未提交数据(脏数据)

需要将隔离级别设置成 未提交读(read-uncommitted)

窗口1

```
set autocommit=0
```

```
autocommit=0
```

```
set session transaction isolation level read uncommitted;
```

```
transaction isolation level read uncommitted;
开启事务(start transaction;)
务(start transaction;)
```

窗口2

```
set
```

开启事

```
update t_account
```

```
set balance=800 where id=1;
```

```
select * from t_account where id=1; 发现我们能查询到别人没有提交数据
```

- read-committed

解决脏读：将隔离级别设置为read-committed

```
set session transaction isolation level read committed;
```

将自动提交要关闭

但是出现了不可重复读：

2、演示不可重复读 一个事务 读到 另外一个事务提交数据
需要将隔离级别设置成 未提交读(read-uncommitted)

窗口1

窗口2

<code>set autocommit=0</code>	<code>set</code>
<code>autocommit=0</code>	
<code>set session transaction isolation level read committed;</code>	<code>set session</code>
<code>transaction isolation level read committed;</code>	
开启事务(start transaction;)	开启事
务(start transaction;)	
	<code>update t_account</code>
<code>set balance=800 where id=1;</code>	
	<code>commit;</code>
 <code>select * from t_account where id=1;</code> 发现我们能查询到别人没有提交数据	

- repeatable read

解决不可重复读：将隔离级别设置为repeatable read

```
set session transaction isolation level repeatable read;
```

将自动提交要关闭

但是出现了不可重复读：

3、演示幻读

需要将隔离级别设置成 未提交读(repeatable read)

窗口1

窗口2

<code>set autocommit=0</code>	<code>set</code>
<code>autocommit=0</code>	
<code>set session transaction isolation level repeatable read;</code>	<code>set session</code>
<code>transaction isolation level repeatable read;</code>	
开启事务(start transaction;)	开启事
务(start transaction;)	
	<code>insert</code>
<code>t_account(id,name,balance) values(3,'ww',1000);</code>	
	<code>commit;</code>
 <code>select * from t_account;</code> 发现没有id=3这条数据	
<code>insert t_account(id,name,balance) values(3,'ww',1000);</code> 报错	

- serializable

解决幻读:

```
set session transaction isolation level serializable;
```

8.Jdbc控制事务

8.1 java中事务的控制

```
conn.setAutoCommit(false) == start transaction;
```

```
conn.commit() == commit;
```

```
conn.rollback(); == rollback;
```

事务步骤（同一事务包裹一系列事情）：关闭自动连接，结束之后提交，异常回滚事务



```
public class AccountDao {

    public static void main(String[] args) {

        //1. 获取链接
        Connection conn = JdbcUtils.getConnection();
        PreparedStatement st=null;
        ResultSet rs=null;

        try {

            //2. 开启事务
            conn.setAutoCommit(false);

            String sql1="update account set money=money-1000 where id=1";

            st=conn.prepareStatement(sql1);
            st.executeUpdate();

            int x=1/0;

            String sql2="update account set money=money+1000 where id=2";

            st=conn.prepareStatement(sql2);
            st.executeUpdate();

            //3. 提交事务
            conn.commit();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        //关闭事务
        try {
            //回滚事务

            conn.commit();
        } catch (SQLException e1) {
            e1.printStackTrace();
            conn.rollback();
        }

    }finally {
        JdbcUtils.release(conn, st, rs);
    }

}

}

```

8.2 事务的回滚点

回滚到指定位置

```
savepoint = conn.setSavepoint();
```

```

public static void main(String[] args) {

    //1.获取链接
    Connection conn = JdbcUtils.getConnection();
    PreparedStatement st=null;
    ResultSet rs=null;

    Savepoint savepoint=null;

    try {

        //2.开启事务
        conn.setAutoCommit(false);

        String sql1="update account set money=money-1000 where id=1";

        st=conn.prepareStatement(sql1);
        st.executeUpdate();

        //设置一个事务的回滚点
        savepoint = conn.setSavepoint();

        int x=1/0;

        String sql2="update account set money=money+1000 where id=2";
    }
}

```

```
st=conn.prepareStatement(sql2);
st.executeUpdate();

//3.提交事务
conn.commit();

}catch (Exception e) {
    e.printStackTrace();

    //关闭事务
    try {
        //回滚事务

        conn.rollback(savepoint);//回滚事务到指定的位置
        conn.commit();
    } catch (SQLException e1) {
        e1.printStackTrace();
    }

}finally {
    jdbcUtils.release(conn, st, rs);
}

}
```